1 Configuring the Core: SCB, NVIC, and SysTick

The behaviour of the exception model is configured through memory-mapped registers in the System Control Space. All of them are accessible only from privileged code.

1.1 System Control Block Registers

Register	Address	Role
`CPUID`	`0xE000_ED00`	Core identification (`0x410FC231` : ARM, Cortex-M3)
`ICSR`	`0xE000_ED04`	Pending state of system exceptions; `PENDSVSET` (bit 28) raises PendSV
`VTOR`	`0xE000_ED08`	Address of the vector table
`CCR`	`0xE000_ED14`	Which conditions are treated as faults
`SHPR1` – `SHPR3`	`0xE000_ED18` – `0xE000_ED20`	Priorities of system exceptions
`SHCSR`	`0xE000_ED24`	Enable and status of system handlers
`CFSR`	`0xE000_ED28`	Cause of MemManage, BusFault, UsageFault
`HFSR`	`0xE000_ED2C`	Cause of HardFault
`MMFAR`	`0xE000_ED34`	Address that caused a MemManage fault
`BFAR`	`0xE000_ED38`	Address that caused a BusFault

`VTOR`. On an exception the CPU reads the handler address from `VTOR + 4 × N`. Bits 31–7 hold the table address, which must be aligned to at least 128 bytes (more when many IRQs exist). Writing `VTOR` is redundant when the table is already at 0, but it makes the boot code correct wherever the image is placed: a bootloader at address 0 can start an application stored higher in Flash, which then points `VTOR` at its own table, or a program can copy its table into RAM to replace handlers at run time.

`SHCSR`. Bits 16 (`MEMFAULTENA`), 17 (`BUSFAULTENA`), and 18 (`USGFAULTENA`) enable the dedicated fault handlers. At reset they are 0, so every fault escalates to HardFault and a single handler cannot tell what went wrong. The other bits report whether handlers are active or pending, so the register is updated with read-modify-write (`|= 0x00070000`), never overwritten.

`CCR`. The configuration bits relevant here are listed below.

Bit	Name	Effect when set
3	`UNALIGN_TRP`	Unaligned word and halfword accesses raise a UsageFault
4	`DIV_0_TRP`	Integer division by zero raises a UsageFault
9	`STKALIGN`	8-byte stack alignment on exception entry (1 at reset)

Without `DIV_0_TRP` , `sdiv` and `udiv` silently return 0 when the divisor is 0 and the program continues with a wrong value; setting the bit turns this silent bug into a fault. Unaligned accesses are legal on the Cortex-M3, only

slower, and trapping them can break correct compiler-generated code, so `UNALIGN_TRP` is usually left off. Since `STKALIGN` is already 1, enabling the division trap requires read-modify-write, giving `0x210`.

`SHPR1` – `SHPR3`. Each system exception has one priority byte, so the three registers can be viewed as a 12-byte array starting at `0xE000_ED18`, indexed by exception number minus 4. Each byte is written with a byte store and needs no read-modify-write.

Register	Byte 0	Byte 1	Byte 2	Byte 3
`SHPR1`	MemManage	BusFault	UsageFault	—
`SHPR2`	—	—	—	SVCall
`SHPR3`	—	—	PendSV	SysTick

A typical policy makes faults most urgent (`0x00`), system calls intermediate (`0x80`), and PendSV and SysTick least urgent (`0xE0`). Reading the words as little-endian, this gives `SHPR2 = 0x80000000` and `SHPR3 = 0xE0E00000`.

1.2 Fault Status Registers

```
void UsageFault_Handler( void )
{
    kprintf( "UsageFault: CFSR=0x%x\n", SCB_CFSR );
    for ( ;; ) { }
}

int main( void )
{
    volatile uint32_t numerator = 42;
    volatile uint32_t zero = 0;
    volatile uint32_t result = numerator / zero;
    (void)result;
    for ( ;; ) { }
}
```

`kprintf` is the kernel's formatted print function, built on the UART driver described in the chapter on devices, interrupts, and system calls; before that driver exists, the same experiment can be followed in gdb by stopping in `UsageFault_Handler` and reading `CFSR`.

The divisor is `volatile`, so the compiler cannot evaluate the division at build time and must emit a real `udiv` instruction. With `CCR.DIV_0_TRP` and `SHCSR.USGFAULTENA` set, the instruction raises a UsageFault, the dedicated handler runs, and it prints `CFSR=0x2000000`: bit 25, `DIVBYZERO`. The handler then loops forever, because a fault is a bug with no sensible way to continue. Removing each configuration step isolates its role:

Configuration	Observed behaviour
`SHCSR` bits and `DIV_0_TRP` set	`UsageFault_Handler` runs and reports the cause

Configuration	Observed behaviour
`SHCSR` bits not set	The fault escalates to HardFault; its kind is lost
`DIV_0_TRP` not set	No fault: the division returns 0 and `main` continues

`CFSR` is the concatenation of three status registers, one per configurable fault. Its most useful bits are:

Bits	Sub-register	Main flags
7–0	MMFSR (MemManage)	`IACCVIOL` (0), `DACCVIOL` (1), `MUNSTKERR` (3), `MSTKERR` (4), `MMARVALID` (7)
15–8	BFSR (BusFault)	`PRECISERR` (9), `IMPRECISERR` (10), `BFARVALID` (15)
31–16	UFSR (UsageFault)	`UNDEFINSTR` (16), `INVSTATE` (17), `UNALIGNED` (24), `DIVBYZERO` (25)

`HFSR` bit 30 (`FORCED`) indicates that the HardFault is an escalated configurable fault; the original cause is then found in `CFSR` . The bits are sticky and are cleared by writing 1 to them.

1.3 NVIC: Device Interrupts

```
NVIC->IPR[UART0TX_IRQn] = 0xC0;                              /* priority of IRQ 1 */
NVIC->ISER[UART0TX_IRQn / 32] = 1UL << ( UART0TX_IRQn % 32 ); /* enable IRQ 1 */
```

Device interrupt requests do not reach the CPU directly: they pass through the NVIC, which numbers them, enables them, and assigns their priority. The first line writes the priority byte of IRQ 1 with a byte store. The second sets bit `n % 32` of word `n / 32` in the enable register; since set and clear registers ignore 0 bits, a plain assignment enables one line without disturbing the others.

Register	Base address	Bit or byte for IRQ n	Write semantics
`ISER`	`0xE000_E100`	Bit `n % 32` of word `n / 32`	1 = enable
`ICER`	`0xE000_E180`	Same	1 = disable
`ISPR`	`0xE000_E200`	Same	1 = set pending
`ICPR`	`0xE000_E280`	Same	1 = clear pending
`IPR`	`0xE000_E400`	Byte n	Priority; lower = more urgent

A device interrupt runs its handler only if all of the following hold:

1. the device is configured to raise it (an interrupt-enable bit in the device's own control register);
2. the NVIC line is enabled in `ISER` ;
3. its priority is more urgent than that of the running code, and `PRIMASK` is 0.

NVIC lines driven by device flags are level-sensitive: as long as the device keeps its interrupt flag set, the request stays active. A handler must therefore *acknowledge* the interrupt at the device, typically by writing 1 to clear the flag, otherwise the handler is re-entered forever as soon as it returns.

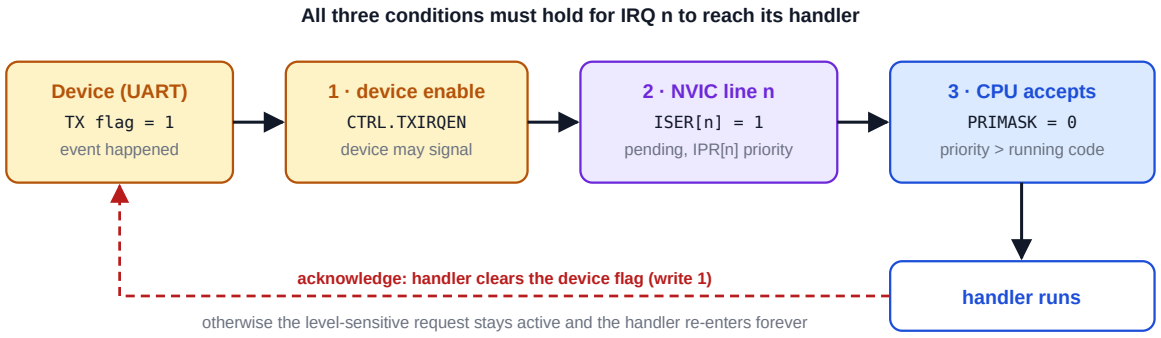


Figure 1.8: *Path of a device interrupt through the device enable, the NVIC, and the CPU masking and priority check, with acknowledgement at the device.*

1.4 SysTick: The Core Timer

SysTick is a 24-bit down-counter present in every Cortex-M core. It counts from the reload value to 0, reloads, and can raise exception 15 each time it reaches 0. An operating system uses this periodic exception, the *tick*, as its time base.

Register	Address	Content
`CTRL`	`0xE000_E010`	`ENABLE` (0), `TICKINT` (1), `CLKSOURCE` (2), `COUNTFLAG` (16, cleared on read)
`LOAD`	`0xE000_E014`	24-bit reload value (max `0xFFFFFF`)
`VAL`	`0xE000_E018`	Current count; any write clears it
`CALIB`	`0xE000_E01C`	Read-only; `TENMS` = reload value for 10 ms on the reference clock

`CLKSOURCE` selects between the processor clock (1) and an implementation-defined reference clock (0). On `mps2-an385` the reference clock runs at 1 MHz (`CALIB.TENMS = 9999`) and the processor clock at 25 MHz. `CALIB` serves portable code that does not know the processor frequency; when the frequency is known, the reload value is computed directly. Writing `CTRL = 7` starts the counter on the processor clock with the exception enabled.

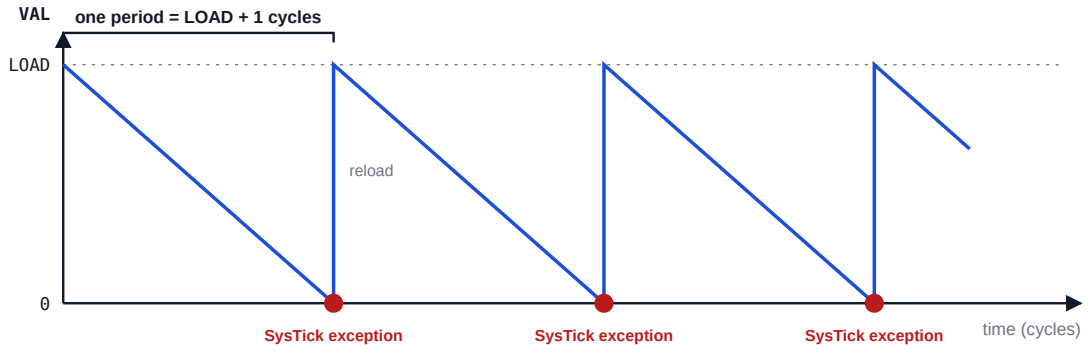


Figure 1.9: *SysTick counter value over time: a sawtooth from LOAD to 0 with an exception at every zero.*

Since one period lasts `LOAD + 1` cycles, the reload value for a tick rate r at clock frequency f is

$$\mathrm{LOAD} = \frac{f}{r} - 1$$

At 25 MHz and 100 ticks per second, `LOAD = 249 999` (`0x3D08F`), which fits in 24 bits; the longest possible period is 2^{24} cycles, about 0.67 s. A faster tick gives finer time resolution at the cost of more exceptions per second.

SysTick is a core exception, not a device interrupt, and this changes how it is configured and handled:

Aspect	SysTick	Device interrupt
Priority set in	`SHPR3`	NVIC `IPR`
Enabled by	`CTRL.TICKINT`	Device enable bit and NVIC `ISER`
Acknowledge in the handler	Not needed: pending state clears on entry	Required, otherwise the handler is re-entered

2 Memory Protection Unit

The Cortex-M3 has no MMU. What it offers instead is an optional **Memory Protection Unit** (MPU), present on `mps2-an385` and disabled after reset.

2.1 MPU Versus MMU

An MMU *translates* every virtual address into a physical one through page tables, and checks permissions as part of the translation. An MPU *only checks*: on every load, store, and instruction fetch, it compares the address with a short list of regions. If the access is allowed, it proceeds at the same physical address; if not, it does not happen at all (a store writes nothing, a load reads nothing) and the CPU raises a MemManage fault. Every task sees real physical addresses, so the linker must place each task at its own address.

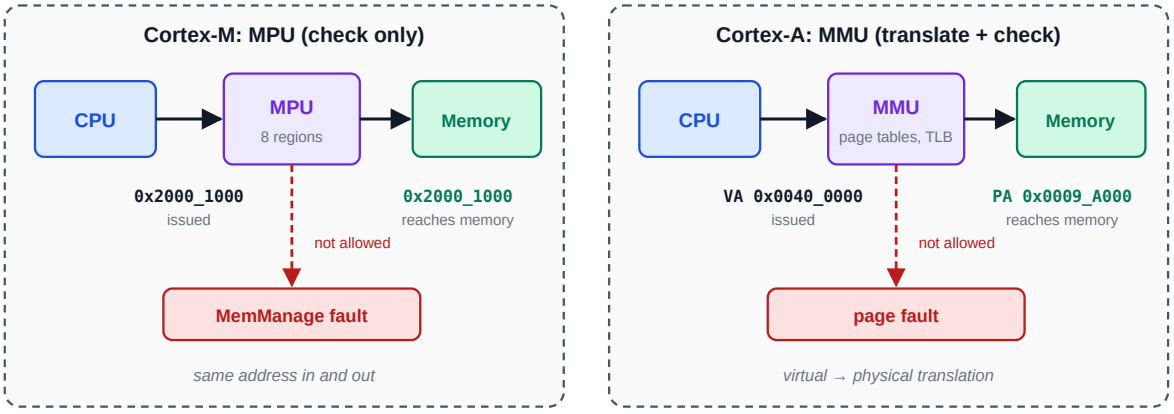


Figure 1.10: *Comparison of an MPU, which only checks addresses, and an MMU, which translates virtual to physical addresses.*

Microcontrollers adopt the MPU for reasons that follow directly from embedded constraints. An MPU is a handful of region registers, while an MMU needs page tables in RAM, a hardware table walker, and a TLB, which cost silicon area and energy. An MPU check takes the same time on every access, whereas a TLB miss makes some

accesses slower by an amount that depends on the TLB contents, which harms timing predictability. Finally, with no disk and a small fixed set of tasks, virtual memory would bring no benefit.

2.2 Regions and Permissions

The Cortex-M3 MPU has 8 regions, numbered 0 to 7 (the count is read from `MPU_TYPE` bits 15–8). Each region has a base address, a size, permissions, and an enable bit, subject to two rules:

- the size is a power of two, from 32 bytes to 4 GB;
- the base address is a multiple of the size, i.e. the region is aligned to its own size.

A 4 KB region can start at `0x2000_1000` but not at `0x2000_0400`. A task that needs 5 KB receives an 8 KB region, the next power of two.

The **AP** (access permission) field encodes two rights at once, one for privileged and one for unprivileged code:

AP	Privileged	Unprivileged
`000`	No access	No access
`001`	Read/write	No access
`010`	Read/write	Read-only
`011`	Read/write	Read/write
`110`	Read-only	Read-only

The **XN** (execute never) bit forbids instruction fetches from a region while still allowing reads and writes if AP permits them. Marking data and stack regions XN prevents code injection: a task cannot execute machine code it has written into a buffer.

The **PRIVDEFENA** bit of `MPU_CTRL` decides what happens to privileged accesses that hit no region. When it is 1, they use the default memory map as if the MPU were off, so the kernel keeps access to all memory and peripherals and only unprivileged code is restricted. When it is 0, even privileged code may access only what the regions allow. An unprivileged access that hits no enabled region is always refused.

2.3 MPU Registers and Region Encoding

Register	Address	Fields
`MPU_TYPE`	`0xE000_ED90`	Bits 15–8: number of regions
`MPU_CTRL`	`0xE000_ED94`	`ENABLE` (0), `PRIVDEFENA` (2)
`MPU_RNR`	`0xE000_ED98`	Region selected by `RBAR` and `RASR`
`MPU_RBAR`	`0xE000_ED9C`	Base address (31–5), `VALID` (4), `REGION` (3–0)
`MPU_RASR`	`0xE000_EDA0`	`XN` (28), `AP` (26–24), `S` / `C` / `B` (18–16), `SIZE` (5–1), `ENABLE` (0)

A single `RBAR` / `RASR` pair serves all eight regions; `RNR` selects which one it refers to. When `RBAR` is written with its `VALID` bit set, the region number is taken from its own `REGION` field, so one write replaces the pair of writes to `RNR` and `RBAR`. The low 5 bits of a base address are always 0, because the base is aligned to at least 32 bytes,

which leaves room for `VALID` and `REGION` in the same word.

The `SIZE` field holds $\log_2(\text{size}) - 1$, so a region spans $2^{\text{SIZE}+1}$ bytes; the offset of 1 lets 5 bits reach 4 GB with `SIZE = 31`. The `S`, `C`, and `B` bits describe the memory type for caches and multi-core systems and have no functional effect on a core without cache.

```
#define MPU_CTRL (*(volatile uint32_t *)0xE000ED94)
#define MPU_RBAR (*(volatile uint32_t *)0xE000ED9C)
#define MPU_RASR (*(volatile uint32_t *)0xE000EDA0)

void mpu_setup( void )
{
    MPU_RBAR = 0x00000000u | 0x10u | 0u;  /* region 0: Flash, VALID */
    MPU_RASR = 0x0602002Bu;               /* AP=110, C, 4 MB, enabled */
    MPU_RBAR = 0x20001000u | 0x10u | 1u;  /* region 1: task partition, VALID */
    MPU_RASR = 0x13060017u;               /* XN, AP=011, S, C, 4 KB, enabled */
    MPU_CTRL = ( 1u << 2 ) | 1u;          /* PRIVDEFENA | ENABLE */
    __asm volatile ( "dsb\n\tisb" : : : "memory" );
}
```

The function configures a typical two-region layout for running one unprivileged task. Region 0 covers the 4 MB of Flash as read-only and executable for everyone, so the task can run its code and read constants. Region 1 covers the 4 KB partition of the running task as read/write and non-executable. Everything else (other tasks' memory, kernel data, kernel stack, peripherals) has no region and is therefore forbidden to unprivileged code, while `PRIVDEFENA` leaves the kernel unrestricted. A kernel that switches tasks only needs to rewrite region 1. The final `dsb` waits for all register writes to complete and `isb` ensures that the following instructions are checked against the new regions.

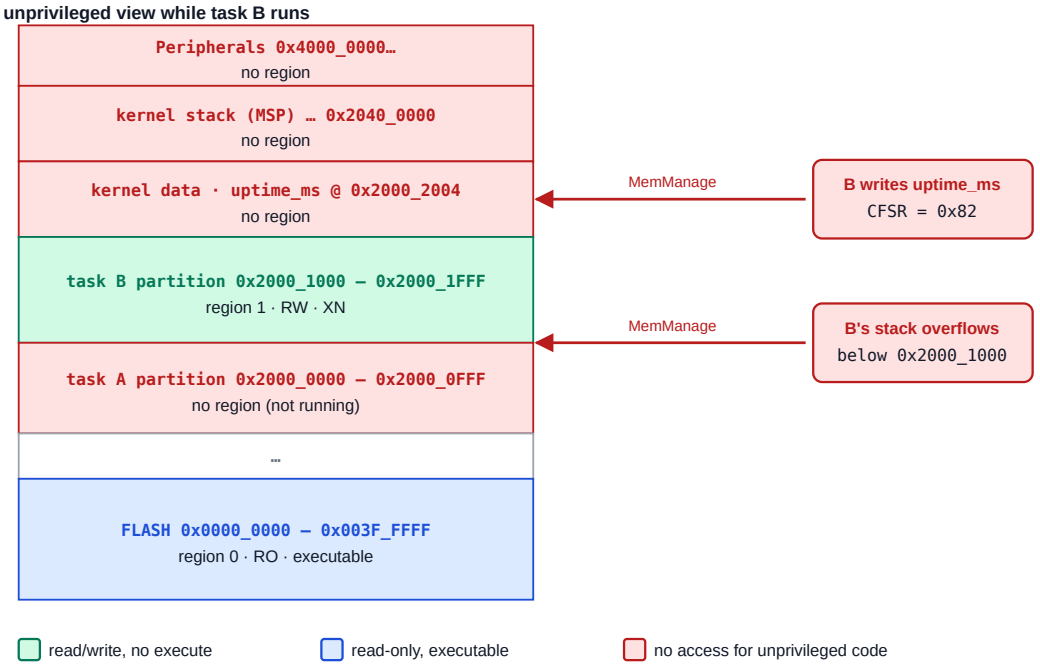
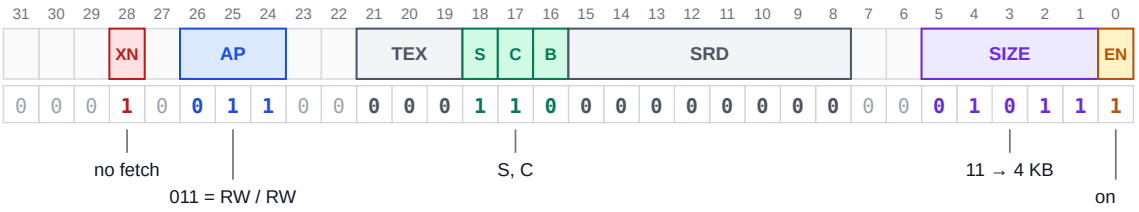


Figure 1.11: *Unprivileged view of memory with the MPU enabled while task B runs: Flash region 0, task B partition region 1, everything else forbidden.*

The two `RASR` values decompose as follows:

Field	Region 0 (Flash)	Region 1 (partition)
`XN` (bit 28)	0	1 → `0x10000000`
`AP` (bits 26–24)	`110` → `0x06000000`	`011` → `0x03000000`
`S`, `C` (bits 18, 17)	`C` → `0x00020000`	`S`, `C` → `0x00060000`
`SIZE` (bits 5–1)	21 (4 MB = 2^{22}) → `0x2A`	11 (4 KB = 2^{12}) → `0x16`
`ENABLE` (bit 0)	`0x01`	`0x01`
Total	`0x0602002B`	`0x13060017`



RASR = 0x1306_0017: region 1 (task partition) · TEX and SRD left at 0

Figure 1.12: *Field layout of MPU_RASR with the value 0x13060017 decoded bit by bit.*

2.4 MemManage Faults

A refused access raises MemManage (exception 4), which reaches its own handler only if `SHCSR.MEMFAULTENA` is set; otherwise it escalates to HardFault. The handler runs privileged and on `MSP`, and it finds the cause in the low byte of `CFSR`:

Bit	Name	Set when
0	`IACCVIOL`	An instruction fetch was refused (e.g. from an XN region)
1	`DACCVIOL`	A data load or store was refused
3	`MUNSTKERR`	Popping the frame on exception return was refused
4	`MSTKERR`	Pushing the frame on exception entry was refused
7	`MMARVALID`	`MMFAR` holds the refused address

Several bits can be set at once. A value of `0x82`, for example, means `MMARVALID` plus `DACCVIOL`: a data access was refused and its address is in `MMFAR`. Looking that address up in the symbol table of the image (with `nm`) names the variable the faulty code tried to touch. A task that overflows its stack below the start of its partition produces the same kind of fault, which makes the MPU an effective stack-overflow detector.

> ⚠ **READ `MMFAR` ONLY WHEN `MMARVALID` IS SET**
>
> For instruction-fetch and stacking faults the CPU does not update `MMFAR`, which may then hold a stale, meaningless value.